

Stockedin

A social investing platform for student / early-career investors -- share real holdings, publish your reasoning, build a verifiable track record, and compare with friends.

OVERVIEW

Flask + SQLite web app. Users create a portfolio of real holdings (ticker, shares, buy price/date), priced live via the FintHub API. Holdings become a social feed post: strategy, risk level, tags, goal and an investment thesis, with likes/comments and a chronological activity log (adds, milestones, predictions). Performance is tracked via daily snapshots rather than self-reported claims.

CORE FUNCTIONALITY

- Accounts & profiles

Signup/login, bio, avatar, optional university/degree/grad year, edit profile.

- Portfolios

Holdings with live pricing; strategy, risk level, goal, tags, and an investment thesis.

- Social graph

Follow/unfollow; mutual-follow-gated direct messages.

- Feed

Followed portfolios plus an activity log: holding adds (with reasoning), auto milestones, predictions.

- Likes & comments

AJAX like/comment on any portfolio post, no page reload.

- Live prices

FintHub quotes + ticker/company-name autocomplete (stocks and ETFs) on the add-holding form.

- Growth tracking

Daily value snapshots; growth chart, tracking duration, and return vs. S&P 500 benchmark.

- Predictions

Make a call (ticker vs. benchmark, by a target date); auto-resolved against live prices, accuracy shown on profile.

- Portfolio personality

Auto-computed labels (Concentrated / Tech Heavy / US Focused / Aggressive Growth) from holdings.

- Leaderboard

Top-moving stocks, most-held stocks, and top-performing portfolios.

- Explore

Trending / Top / Student Investors / Growth / Dividend / AI / Low Risk / New, with searchable cards.

- Student discovery

Find other student investors, filterable by university.

- Groups

Private groups: invite by code or username, member list, group leaderboard, discussion thread.

- News & messaging

Finance news feed (images, load more); mutual-follow-gated DMs; light/dark theme.

BUILT WITH

Python 3.11, Flask, Flask-SQLAlchemy, Flask-Login, Flask-WTF (forms + CSRF), Werkzeug (password hashing), SQLite. FintHub API for live quotes, symbol search and market news. Frontend: Jinja2 templates, vanilla JavaScript (no framework), CSS custom properties for theming, Google Fonts (Inter / Plus Jakarta Sans / Space Grotesk). Pillow for one-off image asset prep; pdf2 for this document.

DEMO WALKTHROUGH (LOG IN AS HUGO / PASSWORD)

1. Home feed: followed portfolios with tags, thesis, holdings and an activity log (adds, milestones, predictions).
2. Open zoe's or heidi's portfolio: growth chart, tracking duration, return vs. S&P 500, auto personality tags.
3. Messages: three pre-populated DM threads (priya, heidi, zoe) -- mutual-follow only.
4. Groups: "AI Infra Club" -- member leaderboard and a live discussion thread.
5. Explore: switch tabs (Trending / Top / Student Investors / Growth / AI / ...).
6. Predictions: hugo has a Pending call; check bob's profile for one already auto-resolved to Correct.
7. Add a holding with a reasoning note -- watch it appear in the activity feed.
8. Follow/unfollow, like, and comment -- all CSRF-protected, still work end to end.
9. Student Investors: filter by university (hugo is at Stanford).
10. Theme toggle (top right) -- full light/dark support.

SECURITY MEASURES

CSRF protection

What: Every state-changing request must carry a signed, session-tied token.

Prevents: Cross-site request forgery -- a malicious page silently submitting actions (follow, delete, post) as a logged-in user.

How: Flask-WTF CSRFProtect, applied globally in `extensions.py/app.py`; a hidden `csrf_token` field on every form, AJAX calls send it as an `X-CSRFToken` header.

Password hashing

What: Passwords are never stored in plain text.

Prevents: Credential exposure if the database is ever read or leaked.

How: Werkzeug `generate_password_hash/check_password_hash` (salted, scrypt-based) in `models.py`.

Login throttling

What: Failed login attempts are rate-limited per identifier + IP.

Prevents: Automated brute-force / credential-stuffing against the login form.

How: In-memory counter (`login_throttle.py`); 5 failed attempts locks that identifier+IP pair out for 60 seconds.

Open-redirect prevention

What: The post-login `?next=` redirect only follows same-site relative paths.

Prevents: Phishing via a crafted login link that bounces the user to an attacker's site after auth.

How: Rejects any next value that isn't a relative path, including protocol-relative `//host` tricks (`routes_auth.py`).

Session cookie hardening

What: The session cookie is locked down beyond Flask's defaults.

Prevents: Cookie theft via JavaScript (XSS) and cross-site cookie leakage.

How: `HttpOnly` + `SameSite=Lax` always on; `Secure` flag enabled via `env var` once served over HTTPS (`config.py`).

Authorization / ownership checks

What: Every mutating route re-checks that the acting user actually owns the resource.

Prevents: IDOR -- one user editing or deleting another user's holdings, portfolio, or group.

How: Explicit id-match checks (e.g. `holding.portfolio.user_id == current_user.id`) before any write, 403 otherwise.

SQL injection prevention

What: All database access goes through the SQLAlchemy ORM query builder.

Prevents: SQL injection via crafted input in forms or query parameters.

How: No raw/string-interpolated SQL anywhere in the codebase; all queries are parameterized by the ORM.

XSS prevention

What: All user-generated content is HTML-escaped on render.

Prevents: Stored/reflected cross-site scripting via bios, comments, theses, or messages.

How: Jinja2's default autoescaping is left on everywhere; no `|safe` filters are used on user input.

SECRETS MANAGEMENT

`SECRET_KEY` (session/CSRF signing) and `FINNHUB_API_KEY` are loaded from a local `.env` file via `python-dotenv`, kept out of source defaults so they aren't hardcoded into the repository.